

TAREA: PROCESAR UN ARCHIVO DE USUARIOS

Programar desde cero un pequeño flujo de procesamiento de datos formado por un **script Bash** y un **script Python**.

Estructura de carpetas

Trabajad con esta estructura:

```
proyecto/
├── scripts/
│   ├── proceso.sh
│   └── python.py
├── users/
│   ├── extracted/
│   │   └── users.csv
│   └── processed/
└── logs/
```

Qué debe hacer el script Bash

El archivo bash.sh debe:

1. Obtener la fecha actual con formato mes_día_año, por ejemplo 10_06_26.
2. Mostrar esa fecha por pantalla.
3. Copiar ../users/extracted/users.csv en la carpeta ../users/processed/.
4. La copia debe llamarse:

FECHA_processed_users.txt

Ejemplo:

10_06_26_processed_users.txt

5. Ejecutar el script Python python.py.

Qué debe hacer el script Python

El archivo python.py debe:

1. Importar las librerías necesarias para procesar y parsear el archivo .csv
2. Obtener la fecha actual con el mismo formato: mes_día_año.
3. Leer el archivo:

../users/extracted/users.csv

4. Recorrer todas las filas del CSV.
5. Detectar si una fila contiene algún dato ausente.
6. Si una fila tiene un dato ausente, añadirla a una lista de filas rechazadas.

7. Si una fila no tiene datos ausentes, añadirla a una lista de filas importadas.
8. Crear dos archivos dentro de la carpeta logs:

```
FECHA_rejected_rows.txt  
FECHA_imported_rows.txt
```

9. En el archivo de rechazadas debe aparecer:
 - Una primera línea indicando cuántas filas han sido rechazadas.
 - Las filas rechazadas debajo.
10. En el archivo de importadas debe aparecer:
 - Una primera línea indicando cuántas filas han sido importadas correctamente.

Ejemplo de resultado

Si el archivo contiene 10 filas, de las cuales 2 tienen algún valor vacío:

Rejected

```
logs/10_06_26_rejected_rows.txt  
There were 2 rejected rows.  
{Después aparecerían las dos filas rechazadas.}
```

Imported

```
logs/10_06_26_imported_rows.txt  
There were 8 imported rows.
```

Además de los requisitos anteriores, el script `python.py` debe incorporar estas funciones.

11. Crea una clase `User` para representar a cada usuario válido del CSV, debe almacenar estos atributos:
 - `name`
 - `surname`
 - `email`
 - `genre`
 - `birth`
 - `phone`
 - `country`
 - `city`
 - `state`

12. Crear una lista de usuarios válidos (junto con los importados). Cuando una fila no tenga datos ausentes:

- Debe añadirse a la lista de filas importadas.
- Debe crearse un objeto de la clase User con los datos de esa fila.
- El objeto creado debe añadirse a la lista users.

13. Conectar con PostgreSQL. El script debe importar la librería necesaria para conectar con PostgreSQL:

- `import psycopg2`

14. Crear una función llamada:

- `database_init_connection()`
- Esta función debe establecer una conexión con la base de datos PostgreSQL

15. Insertar los usuarios válidos creando una función llamada:

- `database_insertion_query(db_connection, users)`
- Esta función debe:
 1. Recibir una conexión a la base de datos y la lista de usuarios válidos.
 2. Crear un cursor para ejecutar consultas SQL.
 3. Recorrer la lista users.
 4. Insertar cada usuario en la tabla users.
 5. Guardar los cambios en la base de datos mediante commit.
 6. Cerrar el cursor.
 7. Cerrar la conexión con la base de datos.

16. La conexión e inicio de la base de datos la tienen que hacer a través de un contenedor Docker, tienen que iniciar la ejecución del contenedor y comprobar la conexión con la base de datos PostgreSQL

Docker string:

docker run -d -p 5432:5432 --name proyecto0 -e POSTGRES_PASSWORD=1234 postgres

SQL db queries:

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(50) NOT NULL,  
  surname VARCHAR(50) NOT NULL,  
  email VARCHAR(255) NOT NULL,  
  genre VARCHAR(20),  
  birth DATE,  
  phone VARCHAR(20),  
  country VARCHAR(50),  
  city VARCHAR(50),  
  state VARCHAR(50)  
);
```

Condición de finalización

La tarea está terminada cuando, al ejecutar:

`./bash.sh`

ocurre todo el flujo sin errores:

- Se crea la copia del CSV en processed.
- Se ejecuta el script Python.
- Se generan los dos archivos de registro en logs.
- Se crean objetos User para las filas correctas.
- Solo se insertan en PostgreSQL los usuarios sin datos ausentes.
- Las filas rechazadas no se insertan en la base de datos.